

Factory Pattern

Category: Creational Design Pattern

1. The Problem

Consider a transportation service app where users can request different types of vehicles (Car, Bike, Bus). A natural first approach is to create instances directly wherever they're needed.

```
public interface Transport {
    void deliver();
}

public class Car implements Transport {
    @Override
    public void deliver() {
        System.out.println("Delivered by Car");
    }
}

public class Bike implements Transport {
    @Override
    public void deliver() {
        System.out.println("Delivered by Bike");
    }
}

public class TransportService {
    public static void main(String[] args) {
        // Direct create objects
        Transport car = new Car();
        Transport bike = new Bike(); // tight coupling

        // add a bus object
        // Transport bus = new Bus();
    }
}
```

Issues:

- **Tight coupling** — the client code (`TransportService`) is tightly coupled to concrete classes (`Car` , `Bike` , `Bus`); it has to know about and directly instantiate every specific type.
- **Modifying client code to extend** — adding a new transport type (e.g. `Bus` , `Truck`) means going back into `TransportService` and adding more `new` calls, spreading object-creation logic across the codebase instead of keeping it in one place.

2. The Solution — Factory Pattern

Intent: Centralize object-creation logic into a dedicated factory, which decides which concrete class to instantiate — so client code depends only on an abstraction, never on concrete classes directly.

- A `TransportFactory` class contains the logic to create the right `Transport` implementation based on an input (e.g. a string like `"car"` or `"bike"`).
- `TransportService` (the client) simply asks the factory for a `Transport` , without knowing or caring how it's built.

3. Key Idea

- **Delegate instantiation to a dedicated class** — instead of scattering `new` calls for different concrete types across client code, one factory method owns that decision.
- **Program to an interface, not an implementation** — the client only ever holds a `Transport` reference, never a `Car` or `Bike` reference directly.
- Adheres to the **Open/Closed Principle** — new transport types can be added by extending the factory's logic, without touching the client code that consumes `Transport` objects.

4. Variants

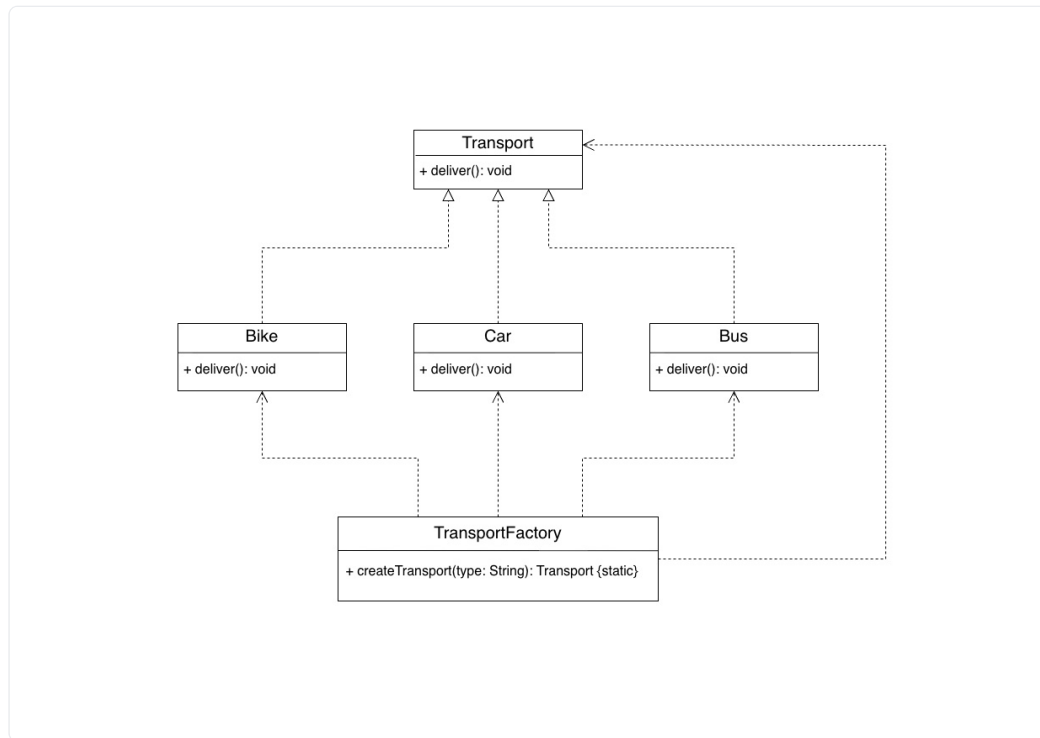
- **Simple/Static Factory** (shown here) — a single static method (`createTransport()`) with a `switch / if-else` that picks the concrete class based on input. Easy to implement, but the factory itself still needs modification whenever a new type is added — it satisfies Open/Closed for the *client*, not for the factory itself.
- **Factory Method Pattern (GoF)** — instead of one static method with a switch statement, each product type gets its own creator subclass overriding a common `createTransport()` method, so even the factory logic can be extended via inheritance rather than modified directly. This is the “true” GoF Factory Method pattern, of which the static factory shown here is a simplified, pragmatic variant commonly used in real codebases.

5. Structure / Participants

Role	Responsibility	Example (from this exercise)
Product (interface)	Declares the common operation(s) all concrete products implement	<code>Transport</code>
Concrete Product	Implements the product interface	<code>Car</code> , <code>Bike</code>
Factory	Centralizes creation logic; decides which concrete product to instantiate	<code>TransportFactory</code>
Client	Requests objects from the factory instead of instantiating them directly	<code>TransportService</code>

Relationship type: Dependency — `TransportService` *depends on* `TransportFactory` to obtain a `Transport` , and `TransportFactory` *depends on* the concrete `Transport` implementations to construct them; the client never depends on the concrete classes directly.

Diagram



Factory Pattern UML diagram

Transport is the product interface declaring `deliver()`, implemented by the concrete products **Bike**, **Car**, and **Bus**. **TransportFactory** depends on all three concrete classes to construct them, and exposes a single static method, `createTransport(type: String): Transport`, which returns the appropriate one — the dashed arrow back to **Transport** shows the factory's return type is the abstraction, not any specific concrete class.

6. Code — Full Implementation

```
public interface Transport {
    void deliver();
}

public class Car implements Transport {
    @Override
    public void deliver() {
        System.out.println("Delivered by Car");
    }
}

public class Bike implements Transport {
    @Override
    public void deliver() {
        System.out.println("Delivered by Bike");
    }
}

public class TransportFactory {

    public static Transport createTransport(String type){
        switch (type.toLowerCase()){
            case "car":
                return new Car();
            case "bike":
                return new Bike();
            case "bus":
                return new Car();
            default:
                throw new IllegalArgumentException("Unsupported Argument");
        }
    }
}

public class TransportService {

    public static void main(String[] args) {

        Transport vehicle = TransportFactory.createTransport("Bike");
        vehicle.deliver();

        vehicle = TransportFactory.createTransport("Car");
        vehicle.deliver();
    }
}
```

What changed vs. the “without pattern” version: - `new Car()` / `new Bike()` called directly inside `TransportService` → both calls now live inside `TransportFactory.createTransport()` . - Client held concrete `Car` / `Bike` variables → client now only ever holds a `Transport` reference, returned by the factory. - Adding a new transport type used to mean adding a new `new` call inside `TransportService` → now it means adding one new `case` inside `TransportFactory` ; `TransportService` is untouched. - Object type is now chosen dynamically at runtime, via a string parameter, instead of being hardcoded at each call site.

7. Benefits

- **Loose Coupling** — the client is decoupled from the specifics of object creation; it only knows the `Transport` interface.
- **Single Responsibility Principle** — the factory class owns the responsibility of object creation, keeping that logic out of the client.
- **Open/Closed Principle** — new transport types can be added without changing the client code, making the system open to extension but closed to modification (from the client's perspective).

8. Drawbacks / Things to Watch Out For

- **The factory itself isn't fully "closed" for modification** in the simple/static variant — every new type still requires editing the `switch` statement inside `TransportFactory`.
- **Can grow into a large switch statement** — as more product types are added, the factory method itself can become long and harder to maintain (the full GoF Factory Method pattern addresses this via subclassing).
- **Slight indirection** — introduces an extra layer/class, which can feel like overhead for systems with only one or two simple product types.

9. Real-World Use Cases

- **GUI Frameworks** — creating platform-specific buttons or widgets (Windows, macOS, Linux) determined at runtime.
- **Database Connectivity** — choosing between different database types (SQL, NoSQL) based on configuration.
- **Document Conversion Tools** — creating the right file handler (PDF, Word, HTML) based on user input or settings.

10. Interview Questions & Answers

Q1. What is the Factory Pattern? A creational design pattern that centralizes object-creation logic into a dedicated factory, which decides which concrete class to instantiate — so client code depends only on a common interface/abstraction, never on concrete classes directly.

Q2. What problem does it solve? It removes the tight coupling that results from client code directly instantiating multiple concrete classes, and avoids having to modify client code every time a new type needs to be supported.

Q3. What are its main components/participants? - **Product (interface)** — the common interface all created objects implement (`Transport`) - **Concrete Product** — a specific implementation (`Car` , `Bike`) - **Factory** — centralizes and owns the creation logic (`TransportFactory`) - **Client** — requests objects from the factory instead of using `new` directly (`TransportService`)

Q4. How does the Factory Pattern support the Open/Closed Principle? Because the client only depends on the `Transport` interface and the factory's method signature, new transport types can be introduced by extending the factory's creation logic (adding a new case, or in the full GoF version, a new creator subclass) — without modifying `TransportService` at all.

Q5. What's the difference between this "Simple/Static Factory" and the full GoF "Factory Method Pattern"? The static factory shown here uses one method with a `switch / if-else` to decide which class to instantiate — simple, but the factory itself must still be edited for every new type. The full GoF Factory Method pattern instead defines an abstract `createTransport()` method that each

product-specific *creator subclass* overrides, so extending the system means adding a new subclass rather than editing existing conditional logic — making even the factory closed for modification.

Q6. How does the Factory Pattern differ from the Builder Pattern? Both are creational patterns, but they solve different problems: **Factory** is about deciding *which concrete class* to instantiate, usually returning a fully-formed object in a single call. **Builder** is about *step-by-step construction* of one complex object that has many (often optional) parts, typically via a fluent chained interface — it isn't primarily about choosing between different types.

Q7. What are the benefits of the Factory Pattern? Loose coupling between client and concrete classes, adherence to the Single Responsibility Principle (creation logic isolated in the factory), and adherence to the Open/Closed Principle from the client's perspective.

Q8. What are the drawbacks? The simple/static factory variant still requires modifying the factory itself for every new type, the factory method can grow into a large conditional block over time, and it adds a small layer of indirection that may be unnecessary for very simple systems.

Q9. Can you give real-world examples? Platform-specific GUI widget creation, choosing a database driver/connection type based on configuration, and document conversion tools that create the appropriate file handler based on the target format.